

Program Repair and Automated Debugging with Code Large Language Models: A Comprehensive Survey of Paradigms, Agents, and Verification

1 Introduction

Automated Program Repair (APR) has historically been framed as a search problem within a constrained space of syntactic mutations, evolving from heuristic generate-and-validate frameworks to sophisticated constraint-solving systems. Early paradigms relied heavily on predefined templates or stochastic mutation operators to navigate the repair search space, often yielding plausible patches that satisfied test suites without guaranteeing semantic correctness [1]. While template-based approaches like TBar demonstrated efficacy by systematizing human fix patterns, they remained inherently limited by the expressivity of their manual rule sets and the brittleness of fault localization prerequisites [2]. Similarly, search-based methods such as Cardumen expanded the exploration horizon through mined templates but struggled with the combinatorial explosion of the search space, frequently producing overfitted solutions that failed to generalize beyond existing test oracles [3].

The advent of deep learning introduced a paradigm shift, recasting repair as a Neural Machine Translation (NMT) task where buggy code is translated into fixed code. Pioneering works like SequenceR and CURE leveraged sequence-to-sequence architectures to learn repair operators directly from data, bypassing the need for explicit template definitions [4; 5]. However, these early neural models often ignored strict syntactic constraints and program semantics, resulting in low compilability rates and a reliance on massive, high-quality training corpora that are costly to curate [6]. Furthermore, the disconnection between the optimization objective (syntactic similarity) and the ultimate goal (functional correctness) led to significant challenges in patch validity, prompting innovations like RewardRepair that integrate execution feedback directly into the loss function [7].

The emergence of Code Large Language Models (Code LLMs) has fundamentally redefined the debugging workflow, transitioning from isolated fault localization to holistic, context-aware generative repair. Unlike their predecessors, modern foundation models such as Code Llama possess an intrinsic understanding of natural language and code semantics, enabling zero-shot and few-shot repair capabilities that transcend specific programming languages [8; 9]. This capability facilitates a conceptual redefinition of debugging where models engage in “rubber duck” reasoning, articulating error causes and synthesizing fixes through chain-of-thought processes rather than mere pattern matching [10]. The integration of LLMs into autonomous agent architectures further amplifies this potential, allowing systems to iteratively refine patches by interacting with compilers, test suites, and static analyzers in a closed feedback loop [11; 12].

Despite these advancements, critical challenges persist regarding the reliability and verifiability of generative repairs. The probabilistic nature of LLMs introduces risks of hallucination and semantic drift, necessitating robust verification mechanisms that go beyond simple test suite passing [13]. Current research increasingly emphasizes neuro-symbolic hybrid systems that combine the generative flexibility of LLMs with the deterministic precision of formal verification and symbolic execution to constrain the search space and ensure correctness [14]. Moreover, the evaluation landscape is shifting from lexical similarity metrics to rigorous assessments of semantic equivalence and maintainability, acknowledging that a correct patch must preserve intended behavior while adhering to human-readable coding standards [15]. This survey synthesizes these evolving paradigms, offering a comprehensive taxonomy of methodologies, agent architectures, and verification strategies that define the state-of-the-art in LLM-driven program repair.

2 Architectural Paradigms and Model Adaptation Strategies

2.1 Prompt Engineering and In-Context Learning Strategies for Repair

Prompt engineering and in-context learning (ICL) represent a paradigmatic shift in automated program repair (APR), enabling pre-trained large language models (LLMs) to synthesize patches without parameter updates. Unlike traditional generate-and-validate frameworks that rely on exhaustive search or template matching, prompt-based approaches leverage the semantic reasoning capabilities of foundation models to navigate the repair space through carefully constructed input contexts. The efficacy of this paradigm hinges on the strategic design of prompts that elicit structured reasoning, optimize contextual relevance, and integrate dynamic feedback loops.

A critical advancement in this domain is the adaptation of Chain-of-Thought (CoT) reasoning to code repair, often manifesting as “self-debugging” or “rubber duck” debugging. By compelling models to articulate logical steps and explain error causes prior to patch generation, these techniques significantly enhance success rates on complex logical defects. [10] demonstrates that few-shot demonstrations of self-explanation enable models to identify mistakes by investigating execution results, achieving state-of-the-art performance on benchmarks like Spider and MBPP. Similarly, [16] teaches models to reason about code execution traces via CoT rationales, improving fix rates by over 26% on program repair tasks. However, standard CoT prompting, designed for natural language, often lacks the structural rigor required for code. [17] addresses this by proposing Structured CoTs (SCoTs), which constrain reasoning to program structures (sequence, branch, loop), thereby outperforming standard CoT by up to 13.79% in Pass@1 metrics.

Contextual optimization is equally pivotal, as the signal-to-noise ratio in input prompts directly impacts repair precision. Strategies such as differential prompting utilize contrastive analysis to isolate root causes. [18] augments conversation-driven APR by providing failing and passing test case pairs, minimizing the difference between them to guide the model toward effective fixes, correctly repairing 143 out of 337 bugs in Defects4J. Furthermore, approaches suggest that presenting buggy and clean code pairs helps models isolate faults more effectively than absolute prediction. The selection of relevant facts, such as stack traces or angelic values, is also crucial; [19] highlights the importance of variable dependency analysis and API documentation in synthesizing precise conditions, achieving a precision of 73.9%.

Iterative refinement frameworks further extend prompt engineering by incorporating compiler errors and test failures as dynamic feedback. This creates a “Synthesize-Execute-Debug” cycle where the model refines hypotheses based on execution traces. [11] implements a fully automated conversation-driven approach that interleaves patch generation with instant feedback, fixing 162 out of 337 bugs at a minimal cost. This iterative process exposes an exploration-exploitation trade-off; [20] frames refinement as an arm-acquiring bandit problem solved via Thompson Sampling, balancing the refinement of high-potential candidates against diverse hypothesis exploration. Additionally, [21] proposes zero-shot verification schemas where models critique their own step-by-step reasoning, mitigating logical errors without external resources.

Despite these advances, challenges remain regarding the reliability of self-correction and the cost of token consumption. [22] notes that performance gains from self-repair are often modest and bottlenecked by the model’s ability to provide accurate feedback on its own code. Moreover, [23] emphasizes the need for cost-efficient prompt designs to minimize token usage while maintaining repair efficacy. Future research must therefore focus on developing more robust verification mechanisms, such as integrating formal constraints into prompts, and optimizing context retrieval to

handle repository-level complexities without exceeding context window limits.

2.2 Supervised Fine-Tuning and Parameter-Efficient Adaptation

Building upon the limitations of static prompting identified previously, Supervised Fine-Tuning (SFT) and parameter-efficient adaptation represent a paradigm shift from eliciting latent capabilities to internalizing specialized program repair patterns directly within model weights. While prompt engineering relies on context-window constraints and inconsistent self-correction, SFT aligns model parameters with the specific syntactic and semantic constraints of Automated Program Repair (APR), creating a more robust foundation for complex debugging tasks. Early approaches treated repair as a Neural Machine Translation (NMT) task, training encoder-decoder architectures on mined buggy-fixed pairs [6]. However, naive translation metaphors often fail to capture the structural nuances of code edits, leading to syntactically invalid outputs [24]. To mitigate this, advanced SFT strategies incorporate code-aware pre-training and search constraints, as demonstrated by [5], which significantly outperforms standard NMT baselines by focusing on compilable patches and subword tokenization.

The construction of high-quality training corpora remains a critical bottleneck, with data contamination posing severe threats to evaluation validity [25]. Recent methodologies have shifted towards self-supervised paradigms to bypass the scarcity of labeled bug-fix commits. For instance, [26] introduces an iterative framework where a “breaker” model generates realistic bugs from clean code, enabling the “fixer” to learn from synthetic yet distribution-aligned data without manual annotation. Similarly, [27] enhances this by perturbing program versions and integrating test execution diagnostics into the input representation, allowing the model to learn project-specific fault types dynamically. Furthermore, [7] proposes reward-driven loss functions that utilize compilation and test execution signals during backpropagation, directly optimizing for semantic correctness rather than mere token-level similarity.

Despite the efficacy of full-model fine-tuning, the computational overhead and risk of catastrophic forgetting necessitate Parameter-Efficient Fine-Tuning (PEFT) strategies. Techniques such as Low-Rank Adaptation (LoRA) allow for the specialization of massive foundation models on domain-specific repair tasks by updating only a small subset of parameters. This approach is particularly vital for adapting generalist models to niche domains like hardware security or legacy code modernization, where full retraining is prohibitive [28]. Moreover, multi-objective fine-tuning frameworks, such as [29], simultaneously optimize for syntactic transformation and the logical reasoning behind code changes, yielding superior repair quality compared to single-objective baselines.

Instruction tuning has further refined these adaptation strategies by structuring repair tasks as conversational or step-by-step reasoning problems. [30] highlights that selecting high-difficulty instruction samples can drastically improve training efficiency, a principle applicable to complex debugging workflows. Additionally, reinforcement learning from execution feedback (RLXF) is emerging as a pivotal alignment mechanism. Frameworks like [31] and [32] employ critic networks and compiler feedback to guide policy optimization, ensuring generated patches satisfy rigorous functional constraints. Similarly, [33] integrates semantic rewards to prioritize security hardening, addressing the tendency of standard SFT to overlook vulnerability patterns. While these parameterized approaches successfully internalize repair heuristics, they remain bound by the static knowledge encoded during training. This limitation motivates the transition to Retrieval-Augmented Generation (RAG), discussed next, which complements parametric memory with dynamic access to external repositories and documentation to handle evolving codebases and long-context dependencies.

2.3 Retrieval-Augmented Generation and Repository-Level Contextualization

Retrieval-Augmented Generation (RAG) has emerged as a critical architectural paradigm for overcoming the inherent context window limitations of Large Language Models (LLMs) in automated program repair. By decoupling knowledge storage from parametric memory, RAG enables models to access repository-specific semantics, historical fix patterns, and external documentation dynamically. This mechanism is essential for grounding generative processes in the broader software ecosystem, thereby mitigating the hallucination of non-existent APIs and ensuring syntactic consistency with project-specific dependencies [34]. The core innovation lies in shifting from static prompting to dynamic context assembly, where retrieved code snippets and fix templates serve as few-shot demonstrations that guide the model toward plausible repair trajectories.

A primary application of RAG in this domain is hybrid patch retrieval, which combines lexical matching with semantic embedding search to identify historically similar bug-fix pairs. Systems like [35] employ contrastive learning to retrieve semantically equivalent bugs from external non-parametric memory, augmenting the generator’s prompt with proven repair strategies. This approach significantly enhances top-1 accuracy by leveraging the “wisdom of crowds” encoded in vast software repositories. Similarly, [36] demonstrates that clustering atomic changes into reusable fix patterns can drastically reduce the search space, yielding patches with a high probability of correctness. However, a critical trade-off exists between retrieval precision and context relevance; overly broad retrieval can introduce noise that distracts the model, while overly narrow retrieval may miss cross-file dependencies essential for complex repairs.

To address repository-level complexity, advanced architectures integrate static analysis tools to resolve cross-file dependencies and API signatures dynamically. [37] illustrates that integrating file-level dependencies during the prompting phase outperforms post-processing corrections, particularly in statically typed languages like Java. This synergy between symbolic analysis and neural generation ensures that retrieved context respects the project’s structural integrity. Furthermore, [38] introduces a mechanism where the LLM learns to trigger autocompletion tools at specific token positions, effectively outsourcing dependency resolution to deterministic engines. This hybrid neuro-symbolic approach reduces undefined-variable errors and improves static validity rates, bridging the gap between probabilistic generation and deterministic correctness.

Beyond code retrieval, dynamic knowledge base integration allows models to query external documentation and issue trackers for up-to-date API guidelines. [39] extends this concept to security, constructing a vulnerability knowledge base from CVE instances to guide reasoning about fixing solutions. By retrieving functional semantics rather than raw code, such systems enable the LLM to reason about vulnerability causes and apply context-aware patches, as seen in [40], which mines role-based access control practices to guide repairs in smart contracts.

Despite these advances, challenges remain in optimizing retrieval latency and handling the “needle in a haystack” problem within massive codebases [41]. Future research must focus on adaptive retrieval strategies that balance the depth of context with computational efficiency, potentially leveraging hierarchical indexing or agent-driven exploration [42]. Ultimately, the convergence of RAG with agentic workflows promises a shift from single-pass generation to iterative, evidence-based repair loops where retrieval and generation co-evolve to solve intricate, multi-file defects.

2.4 Neuro-Symbolic Hybrid Systems and Formal Verification Integration

Building upon the repository-level grounding and static analysis integration established in Retrieval-Augmented Generation (RAG), neuro-symbolic hybrid systems represent a pivotal architectural paradigm that further bridges the probabilistic generative flexibility of Large Language Models (LLMs) with the deterministic rigor of formal methods. While RAG effectively mitigates API hallucinations by retrieving external context, standalone LLMs still frequently suffer from logical hallucinations and a lack of guarantees regarding syntactic validity or logical correctness. Conversely, symbolic engines such as SMT solvers and model checkers offer mathematical precision but struggle with the vast search spaces inherent in real-world software. Hybrid frameworks address these complementary limitations by constraining the LLM’s generation space through symbolic guidance and utilizing the LLM to heuristicize symbolic search processes, creating a robust foundation for the high-assurance guarantees required in safety-critical domains.

A primary integration strategy involves LLM-guided symbolic execution and bounded model checking (BMC). In this architecture, the LLM acts as a heuristic generator for complex artifacts like loop invariants or preconditions, which are subsequently verified by formal engines. For instance, [43] demonstrates a closed-loop system where BMC identifies counterexamples for C program vulnerabilities, which are then fed back to an LLM to synthesize repairs until formal properties are satisfied, achieving an 80% success rate on specific vulnerability classes. Similarly, [44] advances formal verification by employing LLMs to generate whole proofs for theorems in Isabelle/HOL, utilizing error messages from failed proof attempts to iteratively refine the generative output, thereby surpassing traditional search-based techniques. This synergy allows the system to handle infinite-state programs where pure neural approaches fail to ensure termination or safety, establishing a rigorous verification loop that precedes the dynamic runtime feedback mechanisms discussed in agent-centric architectures.

Another critical dimension is the integration of static analysis to enforce syntactic and type constraints during the autoregressive generation process, extending the dependency resolution capabilities seen in RAG systems to the token level. Rather than treating code as a flat token sequence, neuro-symbolic compilers can prune the LLM’s search space in real-time. [45] exemplifies this by coupling an LLM with a completion engine that filters infeasible tokens and proactively suggests valid syntax, significantly increasing the ratio of compilable patches compared to baseline models. Furthermore, [46] incorporates domain-specific rules directly into the decoding procedure via a tree-based architecture, ensuring that generated Abstract Syntax Trees inherently obey semantic constraints. This approach effectively mitigates the generation of syntactically invalid candidates, a common failure mode in pure neural machine translation models like those discussed in [5], and serves as a deterministic guardrail for the generative process.

Beyond generation, neuro-symbolic frameworks enhance validation through execution-guided verification and semantic equivalence checking, addressing the limitations of test-suite-based validation which often leads to overfitting. By incorporating formal specifications or differential testing, these systems provide stronger oracles for patch correctness. [47] proposes a workflow where user intent is formalized into executable tests that act as rigorous oracles for pruning candidate patches. Moreover, [48] introduces an actor-critic framework where an automated reasoning engine evaluates the logical consistency of LLM-generated code, providing feedback loops that significantly improve performance on complex reasoning benchmarks. These systems move beyond simple pattern matching, leveraging symbolic verifiers to ensure that repairs preserve intended behavior across unseen inputs, thereby laying the groundwork for the autonomous, self-correcting loops characteristic of modern

agent systems.

Despite these advances, challenges remain in the latency of symbolic solvers and the difficulty of translating natural language intents into precise formal constraints. Future research must focus on tightening the bi-directional interaction between neural and symbolic components, moving from pipelined architectures to deeply integrated “LLM-Modulo” frameworks where symbolic verifiers actively guide the neural sampling process [49]. Ultimately, the convergence of generative AI with formal verification promises to deliver automated repair systems capable of providing the high-assurance guarantees that serve as the bedrock for the subsequent evolution toward dynamic, agent-centric architectures capable of autonomous planning and tool orchestration.

2.5 Agent-Centric Architectures and Multi-Role Collaboration

The paradigm of automated program repair is undergoing a fundamental transformation from static, single-pass generation to dynamic, agent-centric architectures capable of autonomous planning, tool orchestration, and iterative self-correction. Unlike traditional generate-and-validate frameworks that treat repair as a singular translation task, modern agent systems decompose debugging into a sequence of interdependent sub-tasks—such as fault localization, context retrieval, patch synthesis, and validation—thereby mimicking the cognitive workflows of human developers. This shift addresses the inherent limitations of context windows and the lack of environmental grounding in monolithic models. For instance, [12] introduces a finite state machine-guided agent that autonomously interleaves information gathering with fix validation, demonstrating that dynamic tool invocation significantly outperforms fixed-prompt strategies by adapting to intermediate feedback. Similarly, [50] challenges the necessity of complex agentic loops, proposing that a streamlined, two-phase localization-and-repair pipeline can achieve superior efficiency and effectiveness on repository-level benchmarks, suggesting that the optimal architectural complexity remains an open empirical question contingent on the specific repair domain.

A critical advancement in this domain is the emergence of multi-agent collaboration frameworks that simulate professional software engineering teams through role specialization and consensus mechanisms. By assigning distinct personas—such as coder, reviewer, tester, and security auditor—these systems leverage cross-examination to mitigate hallucinations and enhance patch correctness. [51] exemplifies this approach by deploying a quartet of agents (Manager, Repository Custodian, Developer, and Quality Assurance Engineer) to resolve complex GitHub issues, achieving resolution rates significantly higher than single-model baselines through structured task allocation. This synergy is further explored in [52], which integrates rubber-duck debugging principles into agent interactions, forcing explicit explanation and variable tracking to overcome logic errors that plague single-pass models. Furthermore, [53] formalizes this collaboration by mapping agents to established software process models (e.g., Scrum, Waterfall), demonstrating that embedding agents within recognized engineering workflows improves code quality and exception handling compared to ad-hoc generation.

The efficacy of these architectures relies heavily on the integration of executable feedback loops and sophisticated memory mechanisms. Agents are increasingly empowered to write and execute code to verify hypotheses, as seen in [54], which consolidates actions into a unified executable space, allowing models to dynamically revise strategies based on runtime errors. This “act-think-observe” cycle is crucial for addressing non-deterministic bugs and complex dependency chains. Moreover, the concept of self-evolution allows agents to learn from past trajectories; [55] and [27] illustrate how models can be trained or prompted to refine their own outputs using execution diagnostics, effec-

tively closing the loop between generation and verification without human intervention. However, challenges persist regarding the computational cost and latency of multi-turn agent interactions, as well as the risk of infinite repair loops where agents fail to converge on a solution. Future research must therefore focus on optimizing the trade-off between exploration depth and resource efficiency, potentially through hierarchical planning or hybrid neuro-symbolic constraints that guide agent behavior while preserving generative flexibility. Ultimately, the transition to agent-centric systems represents a move toward holistic, context-aware debugging capable of navigating the intricate landscapes of modern software repositories.

3 Autonomous Agents and Iterative Debugging Workflows

3.1 Agentic Architectures for End-to-End Debugging

The paradigm of agentic architectures for end-to-end debugging represents a fundamental shift from static, single-pass generative models to dynamic, autonomous systems capable of orchestrating complex repair workflows. Unlike traditional approaches that treat program repair as a direct mapping from buggy code to patches, agentic frameworks decompose the debugging lifecycle into discrete, iterative phases: comprehension, navigation, fault localization, and repair synthesis. This structural decomposition allows agents to transcend the context window limitations of large language models (LLMs) by actively interacting with the software environment, thereby addressing repository-level defects that require broad contextual understanding.

A primary architectural distinction exists between sophisticated state-machine-driven agents and simplified, linear “agentless” pipelines. Advanced agents, such as [12], employ a finite state machine to guide the LLM through a non-linear progression of tool invocations, dynamically interleaving information gathering with hypothesis validation. In this model, the agent autonomously decides whether to gather more context, localize the fault further, or attempt a fix based on feedback from compilers and test suites. Similarly, [42] enhances this capability by leveraging abstract syntax tree (AST) representations to perform iterative code search, sharpening the context via spectrum-based fault localization before generating modifications. These systems effectively simulate the cognitive loops of human developers, utilizing tools to navigate large codebases and track key variables beyond the scope of individual methods.

Conversely, recent empirical analyses challenge the necessity of complex autonomy. The [50] framework demonstrates that a simplistic two-phase process—deterministic localization followed by targeted repair—can achieve superior performance and cost-efficiency compared to verbose agentic setups on benchmarks like SWE-bench Lite. This suggests that for many current tasks, the overhead of autonomous planning and tool orchestration may outweigh the benefits, provided that the initial fault localization is sufficiently accurate. However, this “agentless” approach often struggles with multi-hunk bugs or issues requiring deep cross-file reasoning, where the adaptive navigation of agents like [56] proves indispensable. [56] specifically models fault localization as a three-step process of comprehension, navigation, and confirmation, utilizing diverse agents to handle project-level contexts that overwhelm monolithic models.

The integration of repository-level context management remains a critical differentiator. While linear pipelines often rely on static retrieval, agentic architectures enable dynamic context augmentation. For instance, [51] employs a multi-agent synergy involving a “Repository Custodian” and “Developer” to resolve GitHub issues, explicitly addressing the challenge of maintaining existing functionalities while incorporating new code. Furthermore, [52] introduces a unified debugging framework where agent specialization and key variable tracking mimic rubber duck debugging,

significantly improving the handling of complex logic errors and program contexts. These collaborative frameworks highlight a trend toward role-based specialization, where distinct agents manage planning, coding, and quality assurance to mitigate the hallucination risks inherent in single-model approaches.

Ultimately, the choice between monolithic agent designs and streamlined pipelines involves a trade-off between flexibility and efficiency. While [50] resets the baseline for cost-effective repair, the capacity of state-machine-driven agents to handle ambiguity and execute long-horizon reasoning chains positions them as the superior architecture for evolving towards fully autonomous software maintenance. Future research must therefore focus on hybridizing these approaches, potentially embedding lightweight state machines within efficient localization pipelines to balance the robustness of autonomous exploration with the precision of deterministic repair strategies.

3.2 Iterative Refinement and Self-Correction Mechanisms

Building upon the architectural foundations of agentic frameworks and the trade-offs between state-machine-driven autonomy and streamlined pipelines, iterative refinement and self-correction mechanisms represent the cognitive core that animates these structures. These mechanisms transform static code generation into a dynamic “Synthesize-Execute-Debug” loop, leveraging the structural decomposition described previously to actively engage with runtime feedback. Unlike single-pass approaches, systems utilizing these loops employ compiler errors, stack traces, and test failures to progressively eliminate hallucinations and logical flaws. The paradigm shifts from mere pattern matching to active reasoning, where agents engage in “rubber duck” debugging by articulating the rationale behind code changes and identifying inconsistencies without human intervention [10]. This self-explanation capability is critical; studies indicate that forcing models to explain execution traces via Chain-of-Thought reasoning significantly enhances their ability to localize faults and synthesize correct patches, particularly in complex logical scenarios where surface-level syntax is insufficient [16].

The efficacy of such self-correction, however, is heavily contingent on the source and reliability of feedback, a factor that distinguishes robust agentic systems from brittle linear pipelines. While internal self-critique often fails to identify logical errors due to the model’s inherent bias toward its own outputs [57], external validation signals provide the robust grounding necessary for effective refinement. Frameworks like [58] demonstrate that interacting with external tools to validate outputs allows models to amend flaws iteratively. In the context of program repair, this manifests as conversational agents that incorporate test failure information from previous attempts to construct refined prompts, thereby avoiding the repetition of identical errors [59; 11]. By treating repair as a multi-turn dialogue, systems like [11] and [18] utilize contrastive test pairs to isolate root causes, achieving state-of-the-art results by minimizing the semantic distance between failing and passing executions.

Optimizing the trajectory of these refinement loops introduces a fundamental explore-exploit trade-off that directly impacts the efficiency of the agentic workflows discussed earlier. Greedy strategies that relentlessly refine the most promising candidate often converge prematurely to local optima, whereas breadth-first exploration incurs prohibitive computational costs. Recent work frames this challenge as an arm-acquiring bandit problem, employing Thompson Sampling to balance the exploitation of high-probability patches against the exploration of diverse repair hypotheses [20]. This probabilistic approach ensures that agents do not discard potentially correct but initially low-scoring variants, thereby expanding the effective search space. Furthermore, process-based

supervision and reinforcement learning from execution feedback are emerging as vital techniques for training models to select optimal refinement paths. By rewarding not just the final correct patch but also the quality of intermediate explanations and debugging steps, these methods align the agent’s policy with the semantic correctness of the repair rather than superficial syntactic validity [60; 7].

Despite these advances, significant challenges remain regarding the stability of self-correction loops, highlighting the limitations of purely cognitive mechanisms without external grounding. Empirical evidence suggests that without strong verifiers or external ground truth, iterative self-critiquing can degrade performance, leading models to introduce new errors while attempting to fix old ones [61]. Consequently, the future of autonomous debugging lies in hybrid architectures where lightweight self-reflection is gated by rigorous, tool-assisted verification, bridging the gap between the cognitive loops discussed here and the explicit tool orchestration detailed in the subsequent section. As agents evolve, the integration of memory mechanisms to learn from past failure trajectories will be essential to prevent infinite repair cycles and enhance sample efficiency [62]. Ultimately, the transition from reactive patching to proactive, self-evolving debugging workflows marks a pivotal maturation in the field, promising agents that not only fix bugs but also comprehend the underlying semantic intent of the software they maintain, setting the stage for the advanced tool-augmented reasoning paradigms to follow.

3.3 Tool-Augmented Reasoning and Environmental Interaction

The paradigm of tool-augmented reasoning represents a critical evolution in autonomous debugging, shifting agents from passive code generators to active investigators capable of orchestrating external software engineering utilities. Unlike static prompt-engineering approaches that rely solely on parametric knowledge, tool-augmented agents dynamically interact with compilers, debuggers, and version control systems to ground their hypotheses in empirical runtime data. This interaction closes the semantic gap between syntactic correctness and functional validity, enabling agents to navigate complex repair spaces through iterative evidence gathering.

A foundational capability in this domain is the orchestration of static and dynamic analysis tools to validate patches before acceptance. Frameworks such as [35] demonstrate the efficacy of coupling large language models with state-of-the-art static analyzers like Infer, where the model retrieves semantically similar bugs and generates fixes that are immediately verified against formal specifications. Similarly, [45] introduces a synergistic framework where a completion engine acts as a semantic filter, pruning infeasible tokens suggested by the LLM autoregressively to ensure static validity. This integration prevents the propagation of syntactically invalid candidates, a common failure mode in pure generative approaches. Beyond static checks, dynamic interaction is paramount. The [63] framework exemplifies advanced environmental interaction by segmenting programs into basic blocks and tracking intermediate variable states during execution. By feeding these runtime traces back to the model, agents can perform step-by-step verification akin to human breakpoint debugging, significantly enhancing the localization of logical errors in complex data flows.

Furthermore, the automation of test generation serves as a powerful mechanism for expanding the validation oracle. Agents are increasingly tasked with synthesizing diverse inputs to stress-test repaired code, thereby mitigating the risk of overfitting to weak existing test suites. [64] illustrates a coverage-guided approach where agents iteratively dialog with an LLM to target uncovered lines and branches, substantially improving regression test quality. In the realm of security, [65] decouples vulnerability reasoning from tool usage, showing that enhancing models with external knowledge

retrieval and function calling significantly improves the detection and remediation of zero-day vulnerabilities. The ability to query documentation and historical commit logs also grounds agents in project-specific conventions. [34] addresses the issue of API hallucination by iteratively retrieving suitable API references and refining the context, ensuring that generated patches adhere to the repository’s existing interface contracts.

Despite these advances, challenges remain in the efficient orchestration of diverse tools. The latency introduced by multiple tool invocations and the complexity of parsing heterogeneous feedback formats (e.g., stack traces vs. static warning logs) require sophisticated planning algorithms. [66] addresses the cognitive load on agents by transforming diverse tool documentation into concise, unified instructions, thereby improving tool utilization efficiency. Moreover, the trade-off between exploration breadth and verification depth necessitates adaptive strategies; while [12] employs a finite state machine to guide tool selection autonomously, simpler “agentless” pipelines [50] suggest that highly optimized, linear localization-repair workflows can sometimes outperform complex agentic loops in cost-effectiveness. Future research must focus on developing standardized interfaces for tool interaction and robust reward models that balance the computational cost of tool usage against the probability of repair success, ultimately enabling agents to operate with greater autonomy in large-scale, industrial codebases.

3.4 Multi-Agent Collaboration and Role Specialization

Building upon the challenges of tool orchestration and the efficiency trade-offs identified in single-agent systems, the paradigm of multi-agent collaboration represents a critical evolution that shifts automated repair from monolithic generation to distributed, role-specialized workflows. By decomposing the debugging process into distinct cognitive functions—such as planning, coding, testing, and reviewing—multi-agent systems effectively distribute the cognitive load inherent in repository-level defect resolution. This specialization not only facilitates the simultaneous exploration of the repair search space but also enables rigorous cross-examination of generated patches, thereby enhancing both the precision and maintainability of fixes beyond the capabilities of isolated, tool-augmented agents.

Recent frameworks have formalized this division of labor by assigning specific personas to independent LLM instances, creating a structure that mirrors human software engineering teams. For instance, [51] introduces a four-agent architecture comprising a Manager, Repository Custodian, Developer, and Quality Assurance Engineer, where the Manager orchestrates task decomposition while the Custodian retrieves repository-wide context, enabling the system to resolve complex GitHub issues that stump single models. Similarly, [52] leverages agent synergy inspired by “rubber duck” debugging, where specialized agents track key variables and comprehend program context to achieve a correctness rate exceeding 97% on standard benchmarks. These systems demonstrate that explicit role assignment facilitates a more structured reasoning process, allowing agents to focus on narrow sub-tasks such as fault localization or patch synthesis with higher fidelity than a generalist agent attempting end-to-end repair.

The efficacy of these collaborative frameworks often hinges on the communication protocols governing inter-agent dialogue, which have evolved from simple hand-offs to sophisticated iterative debate and consensus mechanisms. [67] illustrates how simulating a code review process between developer and tester agents can increase vulnerability detection F1 scores by over 16% compared to single-role approaches, as conflicting hypotheses are resolved through argumentation. In the domain of competitive programming, [68] replicates the full synthesis cycle through four specialized agents that

recall examples, plan, generate, and debug, achieving state-of-the-art pass@1 rates by iteratively refining code based on internal feedback loops. This collaborative critique is further enhanced in [18], where the interaction between agents utilizes contrastive test case pairs to isolate root causes, demonstrating that multi-turn conversations enriched with specific failure data outperform static prompt engineering.

However, this architectural complexity introduces significant trade-offs regarding computational cost and coordination overhead, echoing the efficiency debates seen in tool-augmented single agents. While [12] demonstrates the viability of autonomous planning and tool invocation, it incurs significant token costs per bug, raising questions about economic scalability. Conversely, [50] challenges the necessity of complex autonomous agents, arguing that a simplified, non-agentic pipeline of localization followed by repair can achieve superior performance at a fraction of the cost on benchmarks like SWE-bench Lite. This tension suggests that optimal multi-agent design requires balancing the depth of collaborative reasoning against the efficiency of execution. Addressing these constraints is essential before extending these collaborative architectures into high-stakes domains; consequently, the field is now pivoting toward adapting these agent frameworks for specialized environments such as hardware design, concurrent systems, and cybersecurity, where standard repair paradigms are insufficient.

3.5 Domain-Specific Agent Adaptations and Security Hardening

While general-purpose autonomous agents have demonstrated proficiency in standard software repair, high-stakes domains such as hardware design, concurrent systems, and cybersecurity necessitate rigorous adaptation to handle unique syntactic constraints, non-deterministic behaviors, and formal security requirements. In the realm of hardware description languages (HDL), agents must transcend textual pattern matching to understand temporal logic and synthesis constraints. Recent frameworks like [69] and [70] exemplify this adaptation by integrating retrieval-augmented generation with iterative feedback loops from EDA tools, enabling agents to autonomously resolve syntax errors and functional mismatches in Verilog code. These systems leverage compiler diagnostics not merely as failure signals but as semantic guides for repair, achieving significant improvements in syntactic correctness over baseline models. Furthermore, the integration of formal verification is critical; approaches such as [71] and [72] demonstrate how agents can generate SystemVerilog Assertions (SVA) to formally verify design properties, effectively closing the loop between generative repair and mathematical proof. This shift from simulation-based validation to formal property checking represents a paradigmatic evolution in hardware debugging agents.

In security-critical applications, the stakes of erroneous repairs are paramount, requiring agents to prioritize vulnerability mitigation alongside functional correctness. Standard generate-and-validate pipelines often fail to address the semantic nuances of security flaws, leading to patches that pass tests but remain exploitable. To counter this, specialized frameworks employ reinforcement learning with semantic rewards to align agent policies with security ontologies. For instance, [33] introduces a mechanism where agents are rewarded not just for compiling code, but for generating explanatory comments that demonstrate an understanding of the underlying vulnerability, thereby reducing the recurrence of similar defects. Similarly, [40] enhances large language models by mining common Role-Based Access Control (RBAC) practices from vast code corpora, guiding the agent to synthesize context-aware patches for smart contracts that strictly adhere to established security patterns. These adaptations illustrate that effective security hardening requires agents to possess explicit knowledge of vulnerability taxonomies, such as those provided by [39], which utilizes knowledge-level retrieval to ground repair decisions in historical CVE data.

Concurrent and distributed systems present distinct challenges due to non-deterministic execution paths and race conditions that are difficult to reproduce and verify. Agents operating in this domain must analyze interleaving traces and employ bounded model checking to infer correct synchronization primitives. The work in [43] highlights the efficacy of combining bounded model checking counterexamples with LLM generative capabilities to iteratively refine patches until formal safety properties are satisfied. This neuro-symbolic synergy ensures that repairs are not only plausible but provably safe against specific classes of concurrency bugs. Despite these advances, significant challenges remain, particularly regarding the scalability of formal verification within agent loops and the scarcity of high-quality training data for niche domains like quantum computing or industrial control systems, as noted in [73]. Future research must focus on developing lightweight verification oracles and cross-domain transfer learning strategies to enable agents to generalize security and correctness guarantees across diverse, specialized environments without prohibitive computational overhead.

4 Evaluation Methodologies, Benchmarks, and Correctness Assurance

4.1 Benchmark Landscapes and Dataset Curation Strategies

The evaluation landscape for Code LLM-based program repair has undergone a paradigmatic shift, evolving from isolated, function-level benchmarks to complex, repository-scale challenges that mirror real-world software engineering workflows. Historically, the field relied heavily on curated datasets like Defects4J, which provided structured Java bugs with associated test suites, establishing a critical baseline for comparing generate-and-validate frameworks [74]. While foundational, these benchmarks often suffer from limited scope, focusing on single-location fixes and failing to capture the inter-procedural dependencies characteristic of modern software systems. Recent advancements have introduced repository-level benchmarks such as SWE-bench, which tasks models with resolving real GitHub issues across multiple files, thereby necessitating broader context awareness and sophisticated planning capabilities [75]. Similarly, DevEval and DevBench align evaluation metrics with real-world code distributions and full development lifecycles, exposing the limitations of models trained solely on synthetic or isolated code snippets [76; 77].

A paramount challenge in this era of massive pre-training corpora is data contamination, where benchmark instances inadvertently leak into training data, inflating performance metrics and obscuring true generalization capabilities. Studies quantifying contamination have revealed substantial overlaps between popular benchmarks and open training corpora, prompting the development of contamination-free evaluation strategies [25]. To mitigate this, researchers are adopting temporal splitting techniques, as seen in LiveCodeBench, which continuously collects new problems from programming contests post-dating model training cutoffs to ensure evaluation validity [78]. Furthermore, the curation of specialized datasets for domain-specific tasks, such as hardware debugging [28] and security vulnerability repair [79], highlights the scarcity of high-quality resources outside mainstream languages like Python and Java.

Dataset construction methodologies have also evolved to address the “overfitting” phenomenon, where patches satisfy existing test suites without correcting the underlying semantic fault. Traditional approaches often relied on weak oracles, leading to plausible but incorrect patches [80]. Modern curation strategies increasingly integrate dynamic execution feedback and formal verification to strengthen evaluation oracles. For instance, SelfAPR leverages test execution diagnostics to create self-supervised training samples that steer models toward semantically correct repairs rather than syntactic matches [27]. Additionally, the synthesis of novel bug patterns via mutation-based

consistency testing offers a mechanism to generate diverse, non-memorized evaluation instances that probe a model’s deep code understanding rather than its recall capacity [81]. Despite these advances, significant biases remain regarding bug severity and fix complexity, necessitating a move toward holistic, multi-dimensional evaluation frameworks that assess not only compilation success but also code maintainability, minimality, and human acceptability. Future research must prioritize the development of dynamic, self-evolving benchmarks that can adapt to the rapid progression of model capabilities while rigorously enforcing data isolation to preserve scientific integrity.

4.2 Multi-Dimensional Metrics for Patch Quality and Naturalness

Building upon the evolution of benchmark design and data curation discussed previously, the metrics used to evaluate patch quality must similarly transcend traditional binary outcomes. Historically, Automated Program Repair (APR) relied on test suite execution, deeming a patch successful solely if it compiled and passed provided tests. However, this paradigm is increasingly recognized as insufficient because test suites often under-specify intended behavior, leading to the acceptance of “plausible” but semantically incorrect patches that overfit the oracle [80]. Consequently, the field is shifting toward multi-dimensional metrics that assess semantic equivalence, code naturalness, and maintainability to ensure holistic patch quality, setting the stage for the specific techniques used to mitigate overfitting detailed in the subsequent section.

A primary dimension of this expanded evaluation framework is semantic equivalence, which verifies that a patch preserves the program’s intended behavior beyond the scope of existing test cases. Since traditional dynamic analysis is limited by the completeness of the test oracle, advanced approaches leverage behavior similarity analysis to compare execution traces of the original and patched programs on both passing and failing inputs [82]. Furthermore, the integration of formal methods, such as symbolic execution and SMT solvers, provides rigorous guarantees for specific defect classes, particularly in conditional statement repairs where precise condition synthesis is critical [19; 83]. Recent work also explores using Large Language Models (LLMs) as evaluators, instructing them to assess functional correctness and alignment with human preferences without requiring explicit test oracles, as demonstrated by metrics like ICE-Score [84].

Beyond functional correctness, code naturalness and maintainability have emerged as critical criteria for developer acceptance. Patches that are syntactically valid but stylistically idiosyncratic often incur high cognitive loads during code review. Research indicates that LLMs inherently prefer correct patches that exhibit higher naturalness, suggesting that probabilistic language models can serve as effective rankers for patch plausibility [85]. However, empirical studies reveal significant stylistic inconsistencies between LLM-generated code and human-written code, necessitating metrics that quantify readability, conciseness, and adherence to project-specific idioms [86]. To address this, representation learning techniques have been developed to embed code changes, enabling machine learning predictors to estimate patch correctness based on deep semantic features rather than superficial token matching [87].

Edit minimality constitutes another essential metric, rooted in the principle that the optimal repair should introduce the least invasive change necessary to resolve the defect. Extensive mining of software repositories suggests that human fixes are typically concise, often involving single-statement modifications [88]. Multi-objective genetic programming approaches, such as ARJA, explicitly formulate repair as a search problem that optimizes for both correctness and simplicity, penalizing massive refactoring in favor of localized fixes [89]. Similarly, template-based systems like TBar and FixMiner leverage mined fix patterns to constrain the search space to minimal, recurrent edit

scripts, thereby increasing the probability of generating human-like repairs [2; 36].

Finally, the ultimate validation of patch quality lies in human-in-the-loop metrics, including developer acceptance rates and time-to-merge statistics. Surveys of software practitioners indicate that trust in automated repairs is contingent not only on correctness but also on the availability of explainable rationales and the speed of generation [90]. Explainable debugging frameworks that align AI reasoning with human scientific debugging processes have shown to improve developer accuracy in judging patch correctness [91]. Future evaluation frameworks must therefore synthesize these dimensions—semantic rigor, stylistic naturalness, minimality, and explainability—moving beyond the narrow confines of test-suite adequacy. While these metrics define the ideals of patch quality, realizing them requires overcoming the persistent challenge of test-suite overfitting, a topic explored in depth in the following section through dynamic oracle enhancement and behavioral similarity analysis.

4.3 Robustness Analysis and Overfitting Mitigation Techniques

Test-suite overfitting remains a paramount challenge in automated program repair, where generated patches satisfy existing test oracles while failing to address root causes or introducing regressions on unseen inputs. This phenomenon arises because test suites often provide an under-specified approximation of the intended program behavior, allowing “plausible” but incorrect patches to pass validation [80]. To mitigate this, contemporary research has shifted from relying solely on static test suites toward dynamic oracle enhancement and behavioral similarity analysis. A pivotal strategy involves expanding the validation scope through automatic test generation. Techniques such as [82] leverage behavior similarity analysis, generating new test inputs to distinguish between correct patches and those that merely exploit test suite weaknesses. By observing that passing tests on original and patched programs should exhibit similar runtime behaviors, this approach successfully filters out over 56% of incorrect patches without sacrificing valid repairs. Furthermore, fuzzing and symbolic execution are increasingly integrated to stress-test candidate patches. [92] demonstrates the efficacy of using Large Language Models (LLMs) as universal fuzzing engines to generate diverse inputs across multiple languages, thereby uncovering bugs that standard unit tests miss. Similarly, [93] combines LLMs with differential testing to construct robust oracles specifically designed to detect subtle semantic deviations in plausibly correct programs.

Beyond oracle strengthening, robustness analysis now encompasses assessing model resilience against adversarial perturbations and context shifts. Overfitting often stems from models learning superficial syntactic patterns rather than deep semantic logic. To counter this, [20] frames the iterative refinement process as an explore-exploit tradeoff, utilizing Thompson Sampling to balance refining high-potential candidates against exploring diverse repair hypotheses, thus reducing the likelihood of converging on brittle, overfitted solutions. Additionally, [18] enhances conversation-based repair by providing contrastive test case pairs—comprising both failing and passing tests—to isolate root causes more precisely, ensuring that fixes generalize beyond the specific failure trigger. The integration of formal methods offers another layer of defense; [43] illustrates a neuro-symbolic pipeline where Bounded Model Checking verifies LLM-generated patches against formal properties, providing mathematical guarantees that transcend test-suite limitations.

However, trade-offs persist between computational cost and verification rigor. While approaches like [19] achieve high precision by mining variable dependencies and API documentation to synthesize exact conditions, they may struggle with scalability compared to lighter-weight heuristic filters.

Moreover, the non-deterministic nature of generative models necessitates statistical significance testing to ensure performance claims are not artifacts of random sampling. Future directions must prioritize the development of dynamic, self-evolving evaluation frameworks that continuously generate adversarial test cases, moving the field toward a paradigm where patch correctness is defined by semantic equivalence and formal satisfaction rather than mere test suite passage.

4.4 Formal Verification and Execution-Guided Correctness Guarantees

Building on the discussion of dynamic oracle enhancement and the persistent trade-offs between computational cost and verification rigor, the integration of formal verification and execution-guided mechanisms represents a critical evolution in evaluating Large Language Model (LLM)-based program repair. This shift moves the paradigm from probabilistic confidence to mathematically grounded assurances, directly addressing the “overfitting” phenomenon where patches satisfy weak test oracles without addressing root causes. While the previous section highlighted the limitations of static test suites, formal methods now provide rigorous guarantees of semantic correctness and safety properties. Recent frameworks have begun to bridge the gap between the generative flexibility of LLMs and the deterministic precision of symbolic reasoning. For instance, [43] demonstrates a synergistic pipeline where Bounded Model Checking (BMC) is employed not only to detect vulnerabilities and generate counterexamples but also to formally verify the correctness of LLM-synthesized patches, achieving high success rates in repairing memory safety errors in C programs. Similarly, [44] advances this frontier in the domain of interactive theorem proving, utilizing LLMs to generate whole proofs for Isabelle/HOL theorems and employing a repair model to iteratively fix failed proof attempts based on error messages, thereby establishing new state-of-the-art results in automated proof synthesis.

Complementing these static guarantees, execution-guided evaluation frameworks refine the repair process by creating closed-loop validation systems that leverage runtime feedback to converge on correct solutions. Unlike static analysis which may yield false positives, execution-based approaches utilize compilers, sanitizers, and dynamic test runners to provide immediate, high-fidelity feedback, effectively operationalizing the dynamic oracle strategies mentioned earlier. [16] emphasizes the necessity of teaching models to reason about execution traces, showing that incorporating variable states and chain-of-thought rationales significantly improves repair rates. This concept is extended in [63], which introduces a granular debugging strategy where LLMs verify program correctness block-by-block against runtime variable values, effectively pinpointing logical deviations that holistic testing might miss. Furthermore, [94] proposes training dedicated verifier models to assess the semantic validity of generated code based on execution results, outperforming heuristic reranking strategies by directly learning the mapping between runtime behavior and correctness.

The trend toward neuro-symbolic hybridization is also evident in specialized domains such as hardware design and declarative specifications, further expanding the scope beyond traditional software repair. [72] illustrates how multi-LLM agents can generate and validate hardware assertions, ensuring that synthesized designs comply with architectural specifications through assertion-based verification. In the realm of declarative languages, [95] reveals that LLMs, when guided by formal solvers, can outperform traditional techniques in repairing Alloy specifications, highlighting the potential of combining neural generation with constraint solving. However, challenges remain regarding the scalability of formal solvers and the computational cost of iterative verification loops, echoing the efficiency concerns raised in the preceding section. Future research must address the efficiency of these hybrid systems, potentially leveraging techniques like [96] to reduce redundant verification steps, and developing lightweight formal abstractions that can be seamlessly integrated

into continuous integration pipelines without prohibitive latency. Ultimately, the fusion of LLMs with formal verification is not merely an evaluation enhancement but a fundamental requirement for deploying autonomous repair agents in safety-critical systems where heuristic correctness is insufficient, laying the necessary groundwork for the complex agentic workflows and trajectory-based assessments discussed in the following section.

4.5 Agentic Workflows and Autonomous Debugging Evaluation

The evaluation of autonomous agentic workflows in program repair necessitates a paradigm shift from static patch correctness metrics to dynamic assessments of planning efficacy, tool orchestration, and iterative reasoning coherence. Unlike single-pass generation models, agentic systems such as [12] and [42] decompose debugging into multi-stage trajectories involving fault localization, context retrieval, and validation, requiring metrics that capture the efficiency of these search processes. A critical dimension of evaluation is the success rate of bug localization within project-level contexts, where agents like [56] demonstrate the ability to navigate large codebases through comprehension and navigation steps, achieving superior top-1 localization rates compared to method-level baselines. However, high localization accuracy does not guarantee repair success; thus, evaluation must also quantify the “tool selection accuracy” and the semantic relevance of retrieved context, distinguishing between agents that effectively leverage compilation errors and those that succumb to context overflow.

Furthermore, the assessment of self-reflection mechanisms requires analyzing the trajectory of error correction over multiple interaction turns. Systems employing conversational repair, such as [11] and [18], utilize test failure feedback to iteratively refine patches, necessitating metrics that measure the rate of convergence and the avoidance of infinite repair loops. Recent studies indicate that while self-repair can enhance performance, it is often bottlenecked by the model’s capacity to generate accurate feedback on its own code, as highlighted in [22]. Consequently, evaluation frameworks must differentiate between genuine semantic debugging and superficial syntactic tweaking by tracking the reduction in semantic distance between consecutive patch versions rather than mere token overlap. The coherence of long-horizon reasoning chains can be formally assessed by defining a success function $S(\tau)$ over a trajectory $\tau = \{s_0, a_0, s_1, \dots, s_T\}$, where s_t represents the state of the codebase and a_t denotes the agent’s action (e.g., tool invocation or code edit). High-performing agents maximize $S(\tau)$ while minimizing the cumulative cost $C(\tau) = \sum_{t=0}^T c(a_t)$, where $c(a_t)$ accounts for token consumption and API latency.

Cost-benefit analysis emerges as a pivotal metric for practical viability, particularly given the substantial computational overhead of agentic workflows. For instance, [12] reports an average cost significantly higher than simpler pipelines, prompting the development of cost-efficient strategies like [23], which optimizes prompt settings to reduce token usage by up to 73% without compromising repair rates. Additionally, the evaluation of multi-agent collaboration frameworks, such as [51] and [68], demands metrics for inter-agent communication coherence and role specialization effectiveness. These systems simulate software engineering teams where distinct agents (e.g., Manager, Developer, QA) negotiate repair strategies; their efficacy is measured not only by the final patch quality but also by the consensus efficiency and the reduction of hallucinated bugs through cross-examination, as observed in [97]. Finally, robustness against misleading issue descriptions and the ability to handle problems lacking exact ground truth patches remain critical challenges. The [50] study underscores that simplistic, non-agentic approaches can sometimes outperform complex agents in cost and performance on benchmarks like SWE-bench, suggesting that future evaluation methodologies must rigorously control for task complexity and explicitly penalize unnecessary architectural verbosity.

Ultimately, a comprehensive evaluation framework for agentic debugging must integrate trajectory analysis, economic efficiency, and collaborative dynamics to accurately reflect the transformative potential of autonomous software maintenance.

5 Specialized Applications and Domain-Specific Challenges

5.1 Security Vulnerability Detection and Automated Remediation

The application of Code Large Language Models (LLMs) to security vulnerability detection and automated remediation represents a paradigm shift from static, rule-based analysis to dynamic, semantic reasoning. Unlike general program repair, which prioritizes functional correctness against test suites, security remediation demands the elimination of specific exploit vectors while preserving system integrity, often in the absence of comprehensive test oracles. Early investigations into zero-shot vulnerability repair revealed that while off-the-shelf LLMs possess impressive syntactic capabilities, they frequently struggle to generate functionally correct patches for real-world security bugs without specialized guidance [13]. This limitation stems from the models’ pre-training on public repositories that may inherently contain vulnerable patterns, necessitating advanced adaptation strategies to align generative outputs with security best practices.

To address the semantic complexity of vulnerabilities such as SQL injection, buffer overflows, and access control flaws, researchers have moved beyond simple prompt engineering toward hybrid neuro-symbolic frameworks. A prominent approach integrates LLMs with formal verification tools to create self-healing loops. For instance, [43] demonstrates a methodology where Bounded Model Checking (BMC) generates counterexamples for C programs, which are then fed back to the LLM to iteratively refine patches until formal security properties are satisfied. Similarly, [98] introduces vulnerability-semantics-guided prompting (VSP), leveraging Chain-of-Thought reasoning to significantly improve F1 accuracy in patching tasks by forcing the model to articulate the root cause before synthesis. These methods mitigate the risk of hallucination by grounding generative capabilities in deterministic verification results.

Domain-specific challenges further complicate automated remediation, particularly in smart contracts and hardware designs where errors can lead to catastrophic financial or physical consequences. In the realm of blockchain, [40] addresses the nuance of access control bugs by mining role-based permission patterns to guide LLMs, achieving repair success rates nearly double those of baseline models. Concurrently, [99] extends these capabilities to Register Transfer Level (RTL) code, showing that ensembles of LLMs can effectively rectify hardware security vulnerabilities that traditional tools like Cirfix miss. However, the efficacy of these models is not uniform; [79] notes that standard LLMs and deep learning-based APR techniques often fail to address specific Common Weakness Enumeration (CWE) types, such as missing cryptographic steps, highlighting the need for domain-specialized fine-tuning.

Emerging trends emphasize the integration of reinforcement learning (RL) and retrieval-augmented generation (RAG) to enhance security hardening. [33] proposes a semantic reward mechanism that penalizes functional correctness without security improvements, effectively training models to prioritize safety measures like null pointer checks. Furthermore, [39] constructs knowledge bases from CVE instances to retrieve relevant vulnerability causes, enabling the LLM to reason analogically rather than relying solely on parametric memory. Despite these advances, critical challenges remain regarding the scalability of formal verification in the loop and the robustness of models against adversarial perturbations designed to bypass security filters [100]. Future research must therefore focus on developing unified evaluation frameworks that assess not only patch correctness but also

the resilience of the repaired code against evolving attack vectors, ensuring that automated remediation tools evolve from experimental prototypes to trustworthy components of the software development lifecycle.

5.2 Hardware Design Verification and Register Transfer Level Debugging

Bridging the gap between security-focused remediation and the broader challenges of software modernization, the adaptation of Large Language Models (LLMs) to Hardware Design Verification and Register Transfer Level (RTL) debugging represents a critical frontier in Electronic Design Automation (EDA). This domain necessitates a paradigm shift from general-purpose code synthesis to domain-specialized reasoning, as hardware design operates under strict concurrency, timing constraints, and synthesizability requirements distinct from the sequential determinism of software. Early applications revealed significant limitations; standard pre-trained models often generate syntactically invalid Verilog or fail to adhere to hardware security best practices, with studies indicating that approximately 60% of LLM-generated RTL designs may harbor Common Weakness Enumerations (CWEs) [101]. To mitigate these deficiencies, recent methodologies have transitioned toward fine-tuning on domain-specific corpora. Benchmarks such as VerilogEval and datasets derived from HDLBits have facilitated the training of specialized models like VeriGen and CodeV, which leverage multi-level summarization to better capture the semantic nuances of hardware descriptions, outperforming generalist models in functional correctness [102; 103; 104].

A predominant trend in RTL debugging is the integration of iterative, feedback-driven loops that couple LLMs with EDA toolchains, mirroring the self-healing architectures observed in security remediation. Frameworks such as AutoChip and RTLFixer exemplify this “Synthesize-Execute-Debug” cycle, where compiler errors and simulation waveforms are fed back into the model to autonomously correct syntax and logic faults. RTLFixer, employing Retrieval-Augmented Generation (RAG) and ReAct prompting, has demonstrated the ability to resolve 98.5% of compilation errors in benchmark datasets, significantly surpassing one-shot generation capabilities [70; 69]. Similarly, the MEIC framework emphasizes making every iteration count by managing the uncertainties inherent in LLM operations, achieving high fix rates for both syntax and functional errors while offering substantial speedups over manual debugging [105]. Beyond syntax, the verification of functional correctness remains paramount. Systems like VeriAssist incorporate self-verification mechanisms where the LLM reasons through testbench results to iteratively refine RTL code, minimizing human intervention [106]. Furthermore, the generation of verification assets has seen innovation through tools like AssertLLM and AutoBench, which utilize multi-LLM collaborations to synthesize SystemVerilog Assertions and comprehensive testbenches directly from natural language specifications, thereby enhancing coverage-directed testing [72; 107].

Despite these advances, challenges persist in addressing high-level synthesis (HLS) optimization and security hardening, areas where the need for precision parallels the constraints found in legacy system migration. The translation of C/C++ to HLS-compatible code requires precise pragma insertion and bit-width optimization, areas where LLM-driven frameworks employing static analysis and RAG have shown promise in reducing manual repair efforts [108]. Moreover, the scarcity of high-quality training data continues to hinder performance, prompting the development of automated data augmentation frameworks that generate aligned natural language-Verilog pairs to bootstrap model training [109]. Looking forward, the field is moving toward neuro-symbolic hybrids that combine the generative flexibility of LLMs with the mathematical rigor of formal property verification. As hardware designs grow in complexity, the synergy between LLMs and formal methods will be essential to ensure not only functional correctness but also robustness against security

vulnerabilities and timing violations, ultimately realizing the vision of fully autonomous, reliable chip design flows [71; 110].

5.3 Legacy Code Modernization and Cross-Language Translation

Legacy code modernization and cross-language translation represent a critical frontier in automated software engineering, where the objective extends beyond syntactic transpilation to the preservation of semantic equivalence across disparate technological ecosystems. The advent of Code Large Language Models (LLMs) has catalyzed a paradigm shift from rule-based transformation engines, which often struggle with idiomatic nuances and deprecated library patterns, to generative approaches capable of reasoning about program intent. Early foundational work established the viability of sequence-to-sequence models for code translation, demonstrating that pre-training on extensive corpora enables models to learn structural mappings between languages without explicit rule specification [111]. However, direct translation often suffers from functional discrepancies, including compilation failures, runtime errors, and logical drift, necessitating iterative refinement frameworks.

To address these limitations, recent methodologies have integrated execution-guided feedback loops into the translation pipeline. The [112] study highlights that a significant portion of translation failures stems from a lack of deep comprehension of source programs and ignored discrepancies in input-output types. In response, frameworks like UniTrans employ an iterative repair mechanism where auto-generated test cases validate translated code, prompting the LLM to correct errors based on execution feedback. This “translate-verify-repair” cycle significantly enhances functional correctness compared to one-shot generation. Similarly, [113] proposes a specialized corrector model trained on the error distributions of general LLMs, effectively acting as a universal patching layer for common migration artifacts such as missing imports or loop boundary errors. These approaches underscore an emerging trend: treating translation not as a static mapping but as a dynamic optimization problem constrained by formal specifications and runtime behavior.

The challenge of modernization is further compounded when migrating monolithic architectures to modular, type-safe ecosystems or when dealing with low-resource legacy languages. In such scenarios, the scarcity of parallel training data necessitates advanced adaptation strategies. [114] introduces a continual learning framework that enables a single model to acquire repair and translation capabilities across multiple languages sequentially, mitigating catastrophic forgetting while handling diverse syntaxes. For domain-specific or obscure languages, knowledge distillation techniques allow specialized student models to inherit repair strategies from high-resource teacher models, facilitating the modernization of systems where labeled data is nonexistent. Furthermore, repository-level context is paramount; tools like [115] frame migration as a planning problem, synthesizing multi-step edit chains that respect inter-file dependencies, a capability essential for large-scale refactoring that single-file translators cannot achieve.

Despite these advances, significant trade-offs remain between automation breadth and semantic fidelity. While LLMs excel at generating idiomatic target code, they are prone to hallucinating non-existent APIs or subtly altering business logic, particularly in complex concurrent systems. The integration of static analysis and formal verification is therefore becoming indispensable to constrain the search space and guarantee safety properties post-migration. Future research must prioritize the development of neuro-symbolic hybrids that combine the generative flexibility of LLMs with the rigorous guarantees of formal methods, ensuring that modernized codebases are not only syntactically valid but also behaviorally equivalent to their legacy predecessors. As the field

evolves, the focus will likely shift from mere translation to holistic system evolution, where LLMs act as autonomous agents capable of navigating the entire software lifecycle from legacy assessment to modern deployment.

5.4 Concurrent, Distributed, and Non-Functional Bug Repair

Bridging the gap between domain-specific hardware constraints and the non-deterministic complexities of concurrent systems, the debugging of distributed architectures represents a critical frontier where legacy modernization principles intersect with rigorous verification needs. Traditional deterministic repair paradigms often fail in these contexts due to exponential state spaces and the prevalence of Heisenbugs—defects that vanish during observation. Recent advancements leverage Code Large Language Models (LLMs) to navigate this complexity by integrating causal analysis and formal verification, mirroring the semantic preservation goals seen in code translation. For instance, [116] proposes a framework that combines statistical debugging with fault injection to pinpoint root causes in intermittent failures, effectively isolating the causal predicates responsible for non-deterministic crashes. This approach aligns with the capabilities of LLMs to interpret execution traces and infer synchronization primitives, moving beyond simple pattern matching to semantic reasoning about thread interleavings.

In the realm of concurrent program verification, the integration of LLMs with bounded model checking (BMC) has emerged as a potent strategy for ensuring safety properties, echoing the neuro-symbolic trends observed in hardware and translation domains. [43] demonstrates a pipeline where BMC generates counterexamples for concurrency violations, which are then fed to an LLM to synthesize corrected code that satisfies formal constraints. This hybrid approach mitigates the hallucination risks inherent in purely generative models by grounding patch synthesis in mathematical proofs of correctness. Furthermore, the challenge of reversing computational steps to locate errors in concurrent flows is addressed by [117], which provides a theoretical foundation for exploring concurrent computations backward while maintaining causal consistency. LLMs can utilize such reversible execution traces to understand the dependency chains leading to deadlocks, thereby generating fixes that preserve liveness properties, a capability essential for the robust system evolution discussed in modernization efforts.

Beyond functional correctness, the repair of non-functional defects—including performance anti-patterns, resource leaks, and termination failures—requires deep reasoning about system states, similar to the optimization problems faced in cross-language migration. [35] highlights the efficacy of coupling static analysis with LLMs to address critical performance and security bugs, utilizing retrieval-augmented generation to find semantically similar fixes for resource management errors. The complexity of these repairs is compounded in distributed environments where consensus failures and network partitions occur. While specific benchmarks for distributed protocol repair remain scarce, the principles outlined in [118] suggest that transforming programs to verify against weak memory models is crucial; LLMs can be adapted to perform such transformations automatically, ensuring that patches are robust across different hardware memory consistency models like TSO or ARM.

However, significant trade-offs persist between automation breadth and verification depth. The computational cost of verifying concurrent patches via model checking can be prohibitive, and LLMs may struggle with the long-range dependencies typical in distributed microservices, a challenge that resonates with the data scarcity issues in emerging domains. [20] frames the iterative refinement of such complex code as a bandit problem, suggesting that adaptive strategies like Thompson

Sampling are necessary to balance exploring diverse synchronization fixes against exploiting known correct patterns. Future research must focus on enhancing the ability of LLMs to reason about global system invariants and to generate patches that are not only locally correct but also globally consistent in distributed settings. The synthesis of causal debugging, formal verification, and adaptive refinement represents a promising trajectory toward autonomous repair, setting the stage for applying these rigorous hybrid methodologies to even more specialized frontiers like quantum computing.

5.5 Quantum Computing and Emerging Domain-Specific Languages

The application of Code Large Language Models (LLMs) to quantum computing and emerging domain-specific languages (DSLs) represents a frontier where the scarcity of training data and the divergence of semantic rules from classical programming pose unique challenges. Unlike general-purpose software engineering, where vast corpora of open-source code exist, quantum programming frameworks such as Qiskit operate within a data-scarce regime, necessitating strategies that transcend standard supervised fine-tuning. The fundamental constraint in this domain is the adherence to quantum mechanical principles—such as unitarity and no-cloning—which cannot be validated through simple compilation but require simulation or formal verification. Research indicates that while LLMs demonstrate proficiency in generating syntactically correct code for established languages, their performance degrades significantly in niche domains without specialized adaptation [119]. Consequently, the paradigm shifts from direct generation to neuro-symbolic hybridization, where LLMs act as heuristic guides constrained by rigorous external verifiers.

In the context of quantum circuit synthesis and debugging, the primary obstacle is the “hallucination” of invalid gate sequences or qubit allocations that violate physical constraints. To mitigate this, recent approaches advocate for retrieval-augmented generation (RAG) pipelines populated with scarce but high-quality quantum textbooks and open-source IP cores, effectively grounding the model in domain-specific knowledge to reduce semantic errors [120]. This strategy mirrors successes in hardware description languages (HDLs), where fine-tuning on specialized datasets derived from textbooks and GitHub repositories has been shown to improve syntactic correctness by over 40% compared to base models [103]. Furthermore, the integration of execution feedback loops is critical; similar to the “Self-Debugging” methodology applied to classical code, quantum repair agents must iteratively refine patches based on simulator outputs rather than relying solely on parametric knowledge [10]. This “generate-simulate-correct” cycle allows models to learn from counterexamples, gradually aligning their outputs with the probabilistic nature of quantum mechanics.

Beyond quantum computing, emerging DSLs for scientific computing and tensor manipulation face analogous data paucity. Here, the transfer of repair strategies from high-resource teacher models to specialized student models via knowledge distillation offers a viable pathway [112]. However, the correctness criteria in these domains often rely on formal assertions rather than unit tests. For instance, in declarative languages like Alloy or TLA+, repair involves satisfying logical inconsistencies identified through bounded exhaustive search. The synergy between LLMs and formal methods is exemplified by frameworks that combine natural language guidance with symbolic solvers to fix specification bugs, a technique increasingly relevant for verifying quantum algorithms where test oracles are computationally expensive to derive [14]. Additionally, the generation of domain-specific testbenches or assertions by LLMs, as seen in hardware verification contexts, provides a mechanism to bootstrap validation suites where none exist [72].

Future directions in this specialized realm must address the explore-exploit tradeoff inherent in iterative refinement under severe data constraints. Algorithms employing Thompson Sampling or similar bandit strategies can optimize the selection of repair hypotheses when simulation feedback is costly [20]. Moreover, the development of synthetic data generation pipelines, leveraging the LLM’s own generative capabilities filtered by interpreters, presents a promising avenue for bootstrapping training corpora for nascent DSLs [121]. Ultimately, the evolution of program repair in quantum and emerging domains hinges on the tight coupling of probabilistic generation with deterministic verification, ensuring that synthesized code is not only syntactically plausible but physically and logically valid.

6 Human-AI Collaboration, Trustworthiness, and Socio-Technical Implications

6.1 Interaction Paradigms and Human-in-the-Loop Debugging Workflows

The integration of Code Large Language Models (LLMs) into debugging workflows has precipitated a fundamental paradigm shift from static, generate-and-validate mechanisms to dynamic, conversational human-AI collaboration. Unlike traditional Automated Program Repair (APR) systems that operate as opaque black boxes, modern interaction paradigms leverage multi-turn dialogue to transform the developer’s role from a primary coder to an overseer and validator. This evolution is epitomized by systems like [11] and [59], which interleave patch generation with immediate validation feedback, allowing the model to learn from previous failures within a single session. By incorporating test failure information and plausible patch variations into the conversational context, these approaches mitigate the repetition of errors common in stateless sampling, effectively turning the debugging process into a collaborative “Synthesize-Execute-Debug” cycle [122].

A critical dimension of this interaction is the alignment of AI reasoning with human cognitive models through explainable rationales. The [91] framework introduces “Automated Scientific Debugging,” where the LLM is prompted to generate hypotheses and interact with debuggers prior to patch synthesis. This approach mirrors the human “scientific method” of debugging, significantly improving developer trust and decision-making accuracy compared to tools that provide only raw code diffs. Similarly, [16] enhances this capability by teaching models to reason about execution traces via chain-of-thought rationales, enabling the AI to articulate the semantic causes of bugs rather than merely suggesting syntactic fixes. Such transparency is vital; empirical studies indicate that while developers often desire explanations to verify patch correctness, the quality of these rationales directly influences the acceptance rate of automated suggestions [90].

Furthermore, the interface design itself shapes the efficacy of human-in-the-loop workflows. The [123] demonstrates that grounding conversational interactions in code context enables extended, multi-turn discussions that uncover capabilities beyond simple autocompletion. Advanced interaction patterns, such as those implemented in [124], utilize “insert expansion” and structured turn-taking to lower conversation barriers, resulting in a fivefold improvement in bug resolution rates compared to standard question-answer interfaces. In educational contexts, frameworks like [125] illustrate how semi-automatic repair can augment human tutors by leveraging historical conversation data to refine defect detection, thereby reducing workload while enhancing student learning outcomes. However, challenges remain regarding the cognitive load imposed by these interactions. While [126] highlights the productivity gains of prompt-less, context-aware assistance, it also notes disparities in how novices and professionals utilize these tools, suggesting that interaction designs must be adaptive to user expertise levels [127].

Despite these advances, the reliance on conversational feedback loops introduces trade-offs between

autonomy and control. Systems like [12] push toward full autonomy by employing finite state machines to orchestrate tool usage, yet they still require human oversight for complex, multi-file defects where context windows may be insufficient. The future of debugging workflows lies in hybrid architectures that balance autonomous agent planning with intuitive human intervention points. As evidenced by [18], providing contrastive test case pairs within the conversation can sharply isolate root causes, suggesting that future interfaces should prioritize the visualization of behavioral divergences over mere code presentation. Ultimately, the transition to proactive AI partners necessitates a re-evaluation of trust metrics, moving beyond test-suite adequacy toward measures of explainability, interaction efficiency, and the preservation of developer intent.

6.2 Trust Dynamics, Reliability Assessment, and Ethical Accountability

Building upon the hybrid architectures and re-evaluated trust metrics proposed at the conclusion of collaborative workflows, establishing trust in autonomous program repair agents necessitates a paradigm shift from probabilistic confidence to rigorous reliability assessment. As Large Language Models (LLMs) transition from assistive conversational partners to autonomous actors, the foundational challenge lies in the inadequacy of test suites as sole oracles; patches that satisfy existing tests often fail to address root causes or introduce subtle regressions, a phenomenon known as patch overfitting [80]. Traditional evaluation metrics, which prioritize compilation success and test passage, are increasingly recognized as insufficient for guaranteeing semantic correctness. Consequently, the field is moving toward hybrid verification frameworks that integrate formal methods, such as symbolic execution and bounded model checking, to provide mathematical guarantees alongside generative capabilities [43]. By leveraging counterexamples derived from formal analysis to guide LLM refinement, systems like ESBMC-AI demonstrate that closing the loop between probabilistic generation and deterministic validation significantly enhances repair precision [43].

The reliability of these systems is further complicated by the propensity of LLMs to hallucinate plausible but incorrect fixes or propagate insecure coding patterns learned from training data, issues that persist even when human oversight is minimized. Empirical studies indicate that while LLMs can fix synthetic vulnerabilities, they often struggle with real-world security defects, frequently generating code that retains logical flaws or introduces new attack vectors [13; 79]. This “plausibility trap” is exacerbated by the models’ tendency to produce verbatim known bugs at rates comparable to correct code, raising critical concerns about the safety of deploying such tools without robust filtering mechanisms [128]. To mitigate these risks, emerging approaches employ multi-agent architectures where distinct “critic” models evaluate generated patches, effectively simulating a peer-review process that reduces the acceptance rate of hallucinated fixes [97; 67]. Furthermore, techniques that utilize contrastive test pairs to isolate root causes have shown superior performance in generating semantically valid patches compared to standard conversational feedback loops [18].

These technical vulnerabilities underscore the critical ethical imperatives of accountability and the necessity for calibrated human oversight, setting the stage for the broader socio-technical shifts discussed subsequently. The deployment of probabilistic models in safety-critical environments introduces a complex liability landscape: when an AI-generated patch introduces a vulnerability, determining responsibility among the model developer, the tool integrator, and the human overseer remains legally ambiguous. Research highlights that developers often over-trust AI suggestions, potentially leading to skill atrophy and reduced scrutiny of automated outputs [129; 130]. Moreover, biases in training data can lead to disparate repair quality across different programming languages or domains, raising equity concerns in software maintenance [131]. Addressing these ethical challenges requires the development of explainable repair rationales that align AI reasoning with human

cognitive models, ensuring that developers can audit the logical basis of suggested fixes rather than accepting them as black-box solutions [90]. Future research must prioritize the creation of calibrated confidence scores and standardized benchmarks that evaluate not just patch validity, but also the long-term maintainability and security posture of AI-repaired code, thereby fostering a symbiotic relationship where human expertise validates and guides autonomous repair capabilities [132; 133].

6.3 Socio-Technical Impacts on Education, Skill Acquisition, and Workforce Evolution

The integration of Large Language Models (LLMs) into program repair and automated debugging precipitates a profound socio-technical shift, fundamentally altering the landscapes of computing education and professional software engineering. This transformation is characterized by a dualistic tension: AI tools function simultaneously as potent pedagogical scaffolds that democratize access to complex debugging strategies and as potential cognitive crutches that risk eroding fundamental problem-solving competencies. In educational contexts, LLMs have demonstrated remarkable capacity to enhance learning outcomes by providing immediate, personalized feedback. Systems like CodeHelp illustrate how guardrailed LLM interactions can support students in large-scale courses without compromising academic integrity, offering on-demand assistance that complements rather than replaces instructor guidance [134]. Furthermore, LLM-generated explanations of code and error messages have been found to be significantly more understandable and accurate than those produced by novices, effectively serving as high-quality exemplars that accelerate the acquisition of mental models [135; 136].

However, this reliance introduces the critical challenge of skill atrophy. Empirical studies indicate that while LLMs excel at generating boilerplate code and resolving syntax errors, beginning programmers often struggle to formulate effective prompts or evaluate the correctness of generated solutions, leading to a disconnect between tool usage and deep conceptual understanding [127]. The risk is not merely dependency but the degradation of “debugging literacy”—the ability to trace execution logic and infer root causes independently. Research suggests that without structured interventions, such as the “teachable agent” paradigm where students debug AI-generated code, the passive consumption of fixes may inhibit the development of resilience and analytical depth essential for expert practice [125]. Consequently, the pedagogical focus must evolve from syntax mastery to higher-order skills involving specification, verification, and the critical assessment of AI outputs.

In the professional domain, the workforce is undergoing a similar evolution, shifting from manual fault localization to roles centered on system architecture, requirement specification, and AI orchestration. The emergence of autonomous agents capable of end-to-end repair, such as RepairAgent and AutoCodeRover, suggests a future where developers act as overseers of semi-autonomous debugging workflows rather than primary coders [12; 42]. This transition redefines productivity metrics; success is increasingly measured by the efficacy of human-AI collaboration and the ability to manage complex, multi-agent systems rather than lines of code written [137]. Yet, this shift demands new competencies. Developers must cultivate the ability to interpret probabilistic repair suggestions, mitigate hallucination risks, and ensure semantic equivalence in ways that traditional testing suites cannot guarantee [22].

Looking forward, the trajectory of workforce evolution hinges on establishing balanced human-AI symbiosis. Future curricula and professional development programs must prioritize “AI-native” debugging skills, emphasizing the synthesis of formal verification methods with generative capabilities

to ensure trustworthiness. As tools become more autonomous, the premium on human expertise will likely migrate toward defining rigorous correctness criteria and ethical boundaries, ensuring that the efficiency gains from automation do not come at the cost of software reliability or foundational engineering intuition [130]. Ultimately, the sustainable integration of these technologies requires a deliberate recalibration of educational goals and professional expectations to foster a workforce capable of leveraging AI as a collaborative partner while maintaining ultimate accountability for software quality.

6.4 Adversarial Robustness, Security Risks, and Safe Deployment Strategies

Complementing the workforce evolution toward AI orchestration and the heightened demand for rigorous correctness criteria outlined previously, the integration of Large Language Models (LLMs) into automated program repair introduces a critical security paradox: while these systems aim to enhance software reliability, they simultaneously expand the attack surface through inherent model vulnerabilities and adversarial exploitation vectors. As developers transition into roles managing semi-autonomous workflows, the primary security concern shifts to the susceptibility of these repair agents to prompt injection and jailbreaking attacks, where malicious actors manipulate input contexts to override safety guardrails. Recent studies demonstrate that automated frameworks can generate semantic jailbreaks with high efficiency, coaxing models into ignoring security constraints or generating vulnerable code through iterative refinement strategies [138; 139]. Furthermore, sophisticated attacks like Tree of Attacks with Pruning (TAP) utilize tree-of-thought reasoning to navigate the prompt search space, successfully bypassing state-of-the-art defenses even in black-box settings [140]. These threats are exacerbated by the lack of fundamental instruction-data separation in current architectures, leaving models vulnerable to indirect command injections where untrusted code comments or documentation alter system behavior [141].

Beyond external manipulation, the deployment of LLM-based repair tools poses significant risks regarding data privacy and the propagation of insecure coding patterns, directly challenging the trustworthiness required for the “AI-native” debugging skills discussed earlier. The reliance on cloud-based APIs for proprietary codebases raises concerns about data leakage and inadvertent training data contamination, challenging the confidentiality of intellectual property [142; 25]. Moreover, empirical evidence suggests that LLMs trained on public repositories often reproduce known vulnerabilities, such as buffer overflows or SQL injection flaws, effectively acting as vectors for recycling historical bugs [128; 79]. This phenomenon is particularly acute in security-critical domains, where models may prioritize syntactic correctness over semantic security, generating patches that pass functional tests but leave underlying vulnerabilities intact [13; 143].

To mitigate these risks and realize the promise of trustworthy human-AI symbiosis, a multi-layered defense strategy is essential, combining robust prompt engineering with formal verification and retrieval-augmented generation (RAG). Defensive mechanisms such as backtranslation and Defensive Prompt Patching (DPP) have shown promise in neutralizing jailbreak attempts by reconstructing original intents and applying interpretable suffix prompts, thereby preserving model utility while reducing attack success rates [144; 145]. Additionally, integrating RAG with curated vulnerability knowledge bases allows models to reason against verified security patterns rather than relying solely on parametric memory, significantly improving detection and repair accuracy [39; 98]. Crucially, the adoption of neuro-symbolic hybrid architectures, where LLM-generated patches are subjected to bounded model checking or static analysis before deployment, provides a necessary deterministic filter to ensure security properties are mathematically satisfied, addressing the limitations of probabilistic suggestions noted in professional practice [43]. Future research must prioritize the de-

velopment of instruction hierarchies that explicitly prioritize privileged security commands over user inputs, alongside the creation of dynamic, self-evolving evaluation frameworks capable of detecting emerging adversarial patterns in real-time [146; 147]. Only through such rigorous, defense-in-depth approaches can the community ensure that AI-driven debugging tools serve as trustworthy allies rather than latent liabilities in the software supply chain.

7 Conclusion

This subsection synthesizes the transformative shift in automated program repair (APR) from rigid, search-based heuristics to the probabilistic, semantic reasoning capabilities of Code Large Language Models (LLMs). The survey elucidates that while traditional generate-and-validate frameworks [1] and template-based approaches [2] established foundational correctness guarantees, they were inherently constrained by limited search spaces and reliance on explicit test suites. In contrast, modern LLM-driven paradigms leverage few-shot learning and chain-of-thought reasoning to navigate complex repair landscapes, achieving superior performance on benchmarks like Defects4J and QuixBugs [148; 149]. However, this transition introduces a critical trade-off: the generative flexibility of LLMs often comes at the expense of deterministic correctness, necessitating robust verification mechanisms to mitigate hallucinations and overfitting [82].

A comparative analysis of architectural strategies reveals that no single approach universally dominates. Prompt engineering and conversational frameworks, such as ChatRepair, effectively utilize compiler feedback to iteratively refine patches without parameter updates [11; 59]. Conversely, supervised fine-tuning on curated bug-fix corpora significantly enhances domain-specific repair patterns, though it risks catastrophic forgetting and data contamination [149; 150]. Emerging agent-centric architectures represent a paradigm shift toward autonomous debugging, where systems like RepairAgent and AgentFL decompose tasks into planning, localization, and validation phases, mimicking human developer workflows [12; 56]. Yet, recent findings suggest that simplified, “agentless” pipelines can sometimes outperform complex multi-agent swarms in cost-efficiency and success rates, challenging the necessity of excessive orchestration complexity [50].

Despite these advancements, critical gaps persist. The field lacks unified theoretical frameworks to formally characterize the search space of LLM-generated patches, and current evaluation metrics remain overly reliant on weak test suites that fail to guarantee semantic equivalence [15; 19]. The prevalence of “plausible but incorrect” patches underscores the urgent need for integrating neuro-symbolic hybrid systems that combine LLM generativity with formal verification and static analysis [35; 45]. Furthermore, the scalability of these solutions is threatened by prohibitive token costs and energy consumption, demanding research into parameter-efficient adaptation and greener inference strategies [23; 151].

Future research must prioritize the development of dynamic, self-evolving evaluation frameworks that move beyond static benchmarks to assess long-term maintainability and security hardening [152; 153]. Ultimately, the trajectory of automated debugging points toward a symbiotic human-AI collaboration, where LLMs act as proactive partners in a “rubber duck” debugging capacity, augmenting rather than replacing human expertise to achieve unprecedented software reliability [154; 90].

References

[1] Mining Software Repair Models for Reasoning on the Search Space of Automated Program Fixing

- [2] TBar Revisiting Template-based Automated Program Repair
- [3] Ultra-Large Repair Search Space with Automatically Mined Templates the Cardumen Mode of Astor
- [4] SequenceR Sequence-to-Sequence Learning for End-to-End Program Repair
- [5] CURE Code-Aware Neural Machine Translation for Automatic Program Repair
- [6] An Empirical Study on Learning Bug-Fixing Patches in the Wild via Neural Machine Translation
- [7] Neural Program Repair with Execution-based Backpropagation
- [8] Code Llama Open Foundation Models for Code
- [9] Repair Is Nearly Generation Multilingual Program Repair with LLMs
- [10] Teaching Large Language Models to Self-Debug
- [11] Keep the Conversation Going Fixing 162 out of 337 bugs for \$0.42 each using ChatGPT
- [12] RepairAgent An Autonomous, LLM-Based Agent for Program Repair
- [13] Examining Zero-Shot Vulnerability Repair with Large Language Models
- [14] Automated Fixing of Programs with Contracts
- [15] A Critical Review of Automatic Patch Generation Learned from Human-Written Patches Essay on the Problem Statement and the Evaluation of Automatic Software Repair
- [16] NExT Teaching Large Language Models to Reason about Code Execution
- [17] Structured Chain-of-Thought Prompting for Code Generation
- [18] ContrastRepair Enhancing Conversation-Based Automated Program Repair via Contrastive Test Case Pairs
- [19] Precise Condition Synthesis for Program Repair
- [20] Code Repair with LLMs gives an Exploration-Exploitation Tradeoff
- [21] SelfCheck Using LLMs to Zero-Shot Check Their Own Step-by-Step Reasoning
- [22] Is Self-Repair a Silver Bullet for Code Generation
- [23] CigaR Cost-efficient Program Repair with LLMs
- [24] Patching as Translation the Data and the Metaphor
- [25] Quantifying Contamination in Evaluating Code Generation Capabilities of Language Models
- [26] Break-It-Fix-It Unsupervised Learning for Program Repair
- [27] SelfAPR Self-supervised Program Repair with Test Execution Diagnostics
- [28] LLM4SecHW Leveraging Domain Specific Large Language Model for Hardware Debugging
- [29] Multi-Objective Fine-Tuning for Enhanced Program Repair with LLMs
- [30] From Quantity to Quality Boosting LLM Performance with Self-Guided Data Selection for Instruction Tuning

- [31] CodeRL Mastering Code Generation through Pretrained Models and Deep Reinforcement Learning
- [32] StepCoder Improve Code Generation with Reinforcement Learning from Compiler Feedback
- [33] LLM-Powered Code Vulnerability Repair with Reinforcement Learning and Semantic Reward
- [34] De-Hallucinator Iterative Grounding for LLM-Based Code Completion
- [35] InferFix End-to-End Program Repair with LLMs
- [36] FixMiner Mining Relevant Fix Patterns for Automated Program Repair
- [37] STALL+: Boosting LLM-based Repository-level Code Completion with Static Analysis
- [38] Teaching Code LLMs to Use Autocompletion Tools in Repository-Level Code Generation
- [39] Vul-RAG: Enhancing LLM-based Vulnerability Detection via Knowledge-level RAG
- [40] ACFIX Guiding LLMs with Mined Common RBAC Practices for Context-Aware Repair of Access Control Vulnerabilities in Smart Contracts
- [41] RepoQA: Evaluating Long Context Code Understanding
- [42] AutoCodeRover Autonomous Program Improvement
- [43] A New Era in Software Security Towards Self-Healing Software via Large Language Models and Formal Verification
- [44] Baldur Whole-Proof Generation and Repair with Large Language Models
- [45] Copiloting the Copilots Fusing Large Language Models with Completion Engines for Automated Program Repair
- [46] KNOD Domain Knowledge Distilled Tree Decoder for Automated Program Repair
- [47] Interactive Code Generation via Test-Driven User-Intent Formalization
- [48] LLM-ARC: Enhancing LLMs with an Automated Reasoning Critic
- [49] LLMs Can't Plan, But Can Help Planning in LLM-Modulo Frameworks
- [50] Agentless: Demystifying LLM-based Software Engineering Agents
- [51] MAGIS LLM-Based Multi-Agent Framework for GitHub Issue Resolution
- [52] A Unified Debugging Approach via LLM-Based Multi-Agent Synergy
- [53] When LLM-based Code Generation Meets the Software Development Process
- [54] Executable Code Actions Elicit Better LLM Agents
- [55] CYCLE Learning to Self-Refine the Code Generation
- [56] AgentFL Scaling LLM-based Fault Localization to Project-Level Context
- [57] When Can LLMs Actually Correct Their Own Mistakes? A Critical Survey of Self-Correction of LLMs
- [58] CRITIC Large Language Models Can Self-Correct with Tool-Interactive Critiquing
- [59] Conversational Automated Program Repair

- [60] Training LLMs to Better Self-Debug and Explain Code
- [61] Can Large Language Models Really Improve by Self-critiquing Their Own Plans
- [62] Self-Edit Fault-Aware Code Editor for Code Generation
- [63] LDB A Large Language Model Debugger via Verifying Runtime Execution Step-by-step
- [64] CoverUp Coverage-Guided LLM-Based Test Generation
- [65] LLM4Vuln A Unified Evaluation Framework for Decoupling and Enhancing LLMs' Vulnerability Reasoning
- [66] EASYTOOL Enhancing LLM-based Agents with Concise Tool Instruction
- [67] Multi-role Consensus through LLMs Discussions for Vulnerability Detection
- [68] MapCoder: Multi-Agent Code Generation for Competitive Problem Solving
- [69] RTLFixer Automatically Fixing RTL Syntax Errors with Large Language Models
- [70] AutoChip Automating HDL Generation Using LLM Feedback
- [71] Using LLMs to Facilitate Formal Verification of RTL
- [72] AssertLLM Generating and Evaluating Hardware Verification Assertions from Design Specifications via Multi-LLMs
- [73] LLM4PLC Harnessing Large Language Models for Verifiable Programming of PLCs in Industrial Control Systems
- [74] Automatic Repair of Real Bugs An Experience Report on the Defects4J Dataset
- [75] SWE-bench Can Language Models Resolve Real-World GitHub Issues
- [76] DevEval: A Manually-Annotated Code Generation Benchmark Aligned with Real-World Code Repositories
- [77] DevBench A Comprehensive Benchmark for Software Development
- [78] LiveCodeBench Holistic and Contamination Free Evaluation of Large Language Models for Code
- [79] How Effective Are Neural Networks for Fixing Security Vulnerabilities
- [80] Automatic Repair of Real Bugs in Java A Large-Scale Experiment on the Defects4J Dataset
- [81] Mutation-based Consistency Testing for Evaluating the Code Understanding Capability of LLMs
- [82] Identifying Patch Correctness in Test-Based Program Repair
- [83] Automatic Repair of Buggy If Conditions and Missing Preconditions with SMT
- [84] ICE-Score Instructing Large Language Models to Evaluate Code
- [85] Practical Program Repair in the Era of Large Pre-trained Language Models
- [86] Beyond Functional Correctness: Investigating Coding Style Inconsistencies in Large Language Models

- [87] Evaluating Representation Learning of Code Changes for Predicting Patch Correctness in Program Repair
- [88] How Often Do Single-Statement Bugs Occur The ManySStuBs4J Dataset
- [89] ARJA Automated Repair of Java Programs via Multi-Objective Genetic Programming
- [90] Trust Enhancement Issues in Program Repair
- [91] Explainable Automated Debugging via Large Language Model-driven Scientific Debugging
- [92] Fuzz4All Universal Fuzzing with Large Language Models
- [93] LLM-Powered Test Case Generation for Detecting Tricky Bugs
- [94] LEVER Learning to Verify Language-to-Code Generation with Execution
- [95] An Empirical Evaluation of Pre-trained Large Language Models for Repairing Declarative Formal Specifications
- [96] Faster Mutation Analysis via Equivalence Modulo States
- [97] LLM Critics Help Catch LLM Bugs
- [98] Chain-of-Thought Prompting of Large Language Models for Discovering and Fixing Software Vulnerabilities
- [99] Fixing Hardware Security Bugs with Large Language Models
- [100] FuzzLLM A Novel and Universal Fuzzing Framework for Proactively Discovering Jailbreak Vulnerabilities in Large Language Models
- [101] All Artificial, Less Intelligence GenAI through the Lens of Formal Verification
- [102] VerilogEval Evaluating Large Language Models for Verilog Code Generation
- [103] VeriGen A Large Language Model for Verilog Code Generation
- [104] CodeV: Empowering LLMs for Verilog Generation through Multi-Level Summarization
- [105] MEIC: Re-thinking RTL Debug Automation using LLMs
- [106] Towards LLM-Powered Verilog RTL Assistant: Self-Verification and Self-Correction
- [107] AutoBench: Automatic Testbench Generation and Evaluation Using LLMs for HDL Design
- [108] Automated C/C++ Program Repair for High-Level Synthesis via Large Language Models
- [109] Data is all you need Finetuning LLMs for Chip Design via an Automated design-data augmentation framework
- [110] LLM for SoC Security A Paradigm Shift
- [111] Unified Pre-training for Program Understanding and Generation
- [112] Exploring and Unleashing the Power of Large Language Models in Automated Code Translation
- [113] Rectifier: Code Translation with Corrector via LLMs
- [114] CIRCLE Continual Repair across Programming Languages

- [115] CodePlan Repository-level Coding using LLMs and Planning
- [116] Causality-Guided Adaptive Interventional Debugging
- [117] Causal-Consistent Reversible Debugging: Improving CauDEr
- [118] Software Verification for Weak Memory via Program Transformation
- [119] Exploring Multi-Lingual Bias of Large Code Models in Code Generation
- [120] HDLdebugger Streamlining HDL debugging with Large Language Models
- [121] Language Models Can Teach Themselves to Program Better
- [122] Fully Autonomous Programming with Large Language Models
- [123] The Programmer’s Assistant Conversational Interaction with a Large Language Model for Software Development
- [124] Exploring Interaction Patterns for Debugging Enhancing Conversational Capabilities of AI-assistants
- [125] CREF: An LLM-based Conversational Software Repair Framework for Programming Tutors
- [126] Using an LLM to Help With Code Understanding
- [127] How Beginning Programmers and Code LLMs (Mis)read Each Other
- [128] Large Language Models and Simple, Stupid Bugs
- [129] Lost at C A User Study on the Security Implications of Large Language Model Code Assistants
- [130] The Robots are Here Navigating the Generative AI Revolution in Computing Education
- [131] An Empirical Study on Usage and Perceptions of LLMs in a Software Engineering Project
- [132] Enhancing Trust in LLM-Generated Code Summaries with Calibrated Confidence Scores
- [133] Automatic Programming: Large Language Models and Beyond
- [134] CodeHelp Using Large Language Models with Guardrails for Scalable Support in Programming Classes
- [135] Comparing Code Explanations Created by Students and Large Language Models
- [136] Using Large Language Models to Enhance Programming Error Messages
- [137] Rocks Coding, Not Development—A Human-Centric, Experimental Evaluation of LLM-Supported SE Tasks
- [138] Jailbreaking Black Box Large Language Models in Twenty Queries
- [139] GPTFUZZER Red Teaming Large Language Models with Auto-Generated Jailbreak Prompts
- [140] Tree of Attacks Jailbreaking Black-Box LLMs Automatically
- [141] Can LLMs Separate Instructions From Data And What Do We Even Mean By That
- [142] The Stack 3 TB of permissively licensed source code
- [143] A Comprehensive Study of the Capabilities of Large Language Models for Vulnerability Detection

- [144] Defending LLMs against Jailbreaking Attacks via Backtranslation
- [145] Defensive Prompt Patch: A Robust and Interpretable Defense of LLMs against Jailbreak Attacks
- [146] The Instruction Hierarchy Training LLMs to Prioritize Privileged Instructions
- [147] See What LLMs Cannot Answer: A Self-Challenge Framework for Uncovering LLM Weaknesses
- [148] An Analysis of the Automatic Bug Fixing Performance of ChatGPT
- [149] Impact of Code Language Models on Automated Program Repair
- [150] Getafix Learning to Fix Bugs Automatically
- [151] Towards Greener LLMs Bringing Energy-Efficiency to the Forefront of LLM Inference
- [152] Evolving Code with A Large Language Model
- [153] Top Leaderboard Ranking = Top Coding Proficiency, Always EvoEval Evolving Coding Benchmarks via LLM
- [154] A Deep Dive into Large Language Models for Automated Bug Localization and Repair